Learning Record — CSCI3235 Midterm I

Aliyev Nurislam, Student ID: 24B030024

1. Course work before the midterm week

Activity	What I did	Evidence
CSCI3235 practice	Completed and defended the RL practice assignment in class	Defended in class
Coursera module defence	Defended my Coursera RL module progress in class	Defended in class

2. Coursera: Reinforcement Learning Specialization

Course 1, *Fundamentals of Reinforcement Learning* (University of Alberta). Completed modules:

Module	Main ideas
An Introduction to Sequential Decision-Making	k-armed bandits, action values, incremental update, ε-greedy exploration
Markov Decision Processes	agent–environment loop, states, actions, rewards, episodic vs continuing tasks, return and discounting
Value Functions & Bellman Equations	state and action values, policies, Bellman equations, optimal values

3. Independent study and project work during the midterm week

- Studied tabular Q-learning (Sutton & Barto, Section 6.5) and applied it in the LagPilot project.
- Studied Apache Kafka consumer groups, consumer lag, rebalancing and the KEDA Kafka scaler, and Little's Law from my *Backend for Highloaded Environment* course.
- Ran the live Kafka demo myself (FastAPI producer, load generator, Q-table agent) and read the agent's decisions in its log.
- Evidence: project repository with commit history; `results/` (figures and CSVs); `demo/sample_run.log`.

4. Reflection

An idea that became clearer. In the bandit module I learned the incremental rule: new estimate = old estimate + step size × (target − old estimate). In this project I saw that Q-learning uses exactly the same rule. The only difference is the target: instead of the reward alone, it is the reward plus γ times the best Q-value of the next state. The MDP and Bellman modules explained why this extra term is needed: an action changes the next state, so its value depends on the future. In my problem this is very concrete. Adding a Kafka consumer starts a rebalance, which stops processing and makes the lag grow in the next steps, so the cost of the action appears only later. Reading the agent's decisions in the live demo log made this loop of state, action, reward and next state feel real.

An idea that is still difficult. I still find it hard to judge when a state is "Markov enough" and how to choose the discretization. My state hides the exact lag and the time of day, and one of my ten training seeds learned a bad policy. I do not yet fully understand why tabular Q-learning becomes unstable in rarely visited states, or how to fix it reliably with a different step size or more training.